

Cloud Computing Project
STaaS: A Cloud-Based Smart Drive

Group Members:

Lovepreet Sidhu - 100345501- lovepreet.sidhu3@student.kpu.ca
Jashanpreet Brar - 100399847 - jashanpreet.brar4@student.kpu.ca
Omair Waheed - 100420289 - omair.waheed@student.kpu.ca

Kwantlen Polytechnic University
INFO 4110 S10: Cloud Computing
Wei Li, Ph.D., P.Eng.

Submission Date: December 5, 2025

1. Project Background, Technology, and Market Analysis	3
1.1 Project Background	3
1.2 Technology Used	3
1.3 Market Analysis	4
2. System Design	5
2.1 Requirements Summary	5
2.2 System Workflow	5
2.3 Key Features	6
2.4 Architecture Pattern	6
3. AWS Infrastructure Summary	7
4. Architecture Summary	11
5. Testing	12
Login/Register Page	12
Dashboard	13
Uploading New File	14
Changing File Name	15
Deleting Folder	16
Deleting File	17
6. Timelines, Resource Allocation & Project Planning	18
6.1 Project Timeline	18
6.2 Resource Allocation	19
7. Summary	20
8. References	20

1. Project Background, Technology, and Market Analysis

1.1 Project Background

STaaS, also known as Storage as a Service, is a subscription-based cloud computing model for storage where vendors offer secure data storage solutions to customers. Instead of maintaining their own storage infrastructure, which includes servers, hard disks, software, and support staff, companies have providers store and manage data in a secure and accessible environment. STaaS solutions are increasingly adopted by businesses seeking scalability, reduced operational cost, and improved data accessibility. The goal of this project is to design and implement a cloud-based smart storage system that automatically classifies files, supports user-configured folder structures, and provides reliable upload and download functionality through a web-based interface.

1.2 Technology Used

To develop this robust and user-friendly STaaS solution, we chose a modern and scalable combination of technologies. Java Spring Boot is used for backend services, providing a flexible, scalable, and secure framework for developing the application. MongoDB was selected as the database solution due to its document-oriented structure, which fits well with our need for storing unstructured data. AWS S3 serves as the primary object storage service, offering high scalability, durability, and availability for large data sets. Lastly, we used a combination of HTML/CSS along with JavaScript to make a single-page application. Together, these technologies provide a reliable foundation for building a cloud-native storage system that supports smooth integration, efficient file handling, and seamless user interaction.

1.3 Market Analysis

The cloud storage market is dominated by major storage providers such as Google Drive, Dropbox, OneDrive, and iCloud, which focus primarily on consumer-level file sharing, synchronization, and collaboration. They offer wide functionality but are not suitable for business needs.

Many businesses require custom storage solutions. Specifically, businesses instead require:

- Full control over their data

- Customizable storage limits

- Integration with internal systems

- Enhanced data privacy and compliance

STaaS targets this niche market by offering a customizable private cloud storage solution suitable for startups, institutions, and enterprises.

2. System Design

2.1 Requirements Summary

This project has several requirements. Users should be able to do the following:

Drag and drop files into the interface, as well as browse their local file system to select files manually.

Create custom rules for folder sorting, and create new folders as needed.

Rely on the system to automatically sort files into folders based on file type.

For example, a plain text file goes into a ".txt" folder, while an MPEG-4 video goes into a ".mp4" folder. Users can choose which folders are used for uploads, and if the system detects a new file type (for example, a TS video), it will identify the MIME type, create the appropriate folder (such as ".ts"), and place the file there.

Search and sort uploaded files. Sorting options include last access time, creation time, file size, and file name. Searching is performed by file name.

Receive clear confirmation messages for uploads and downloads, indicating whether each operation succeeded or failed.

2.2 System Workflow

The system operates as follows:

Users authenticate and manage their account.

Users create folder hierarchies to organize files.

Files are uploaded through the drag-and-drop interface.

The backend generates secure pre-signed S3 URLs for uploads and downloads.

Files are transferred directly to and from S3 using these URLs.

Metadata is stored in MongoDB to support search, sorting, and folder display.

Storage quotas are enforced to prevent misuse or excessive storage consumption.

2.3 Key Features

User authentication and authorization

Folder management with hierarchical structure

Direct S3 file transfers for efficiency

Storage quota enforcement

Secure pre-signed URLs with temporary access

RESTful API architecture

Global exception handling

Docker-based deployment structure

2.4 Architecture Pattern

The system follows a three-tier architecture using a Controller → Service → Repository pattern. Data is transferred using DTOs, with MapStruct handling object mapping to maintain clean separation between layers.

3. AWS Infrastructure Summary

Region: ca-central-1 (Canada Central)

VPC Details

VPC ID: vpc-01978cf588e4cdcda

CIDR Block: 10.0.0.0/16

State: available

DHCP Options: dopt-097d594d4d36a77c3

Subnets

Public Subnet

Subnet ID: subnet-0ed1137d5873610be

CIDR: 10.0.1.0/24

Availability Zone: ca-central-1a

Auto-assign Public IP: Yes

Available IPs: 249

Private Subnet

Subnet ID: subnet-076cb98c769125b0b

CIDR: 10.0.2.0/24

Availability Zone: ca-central-1a

Auto-assign Public IP: No

Available IPs: 250

Internet Gateway

Internet Connectivity

IGW ID: igw-06376796534dc0e0c

Status: attached to VPC

NAT Gateway

NAT Gateway ID: nat-003e871c07c9e42de

Subnet: subnet-0ed1137d5873610be (Public)

Public IP: 15.157.230.103

Private IP: 10.0.1.16

State: available

Route Tables

Public Route Table

Route Table ID: rtb-02e2559c3f5cc72ff

Associated Subnet: subnet-0ed1137d5873610be

Routes:

10.0.0.0/16 → local

0.0.0.0/0 → igw-06376796534dc0e0c

Private Route Table

Route Table ID: rtb-0380a0fc6a8402fa8

Associated Subnet: subnet-076cb98c769125b0b

Routes:

10.0.0.0/16 → local

0.0.0.0/0 → nat-003e871c07c9e42de

Main Route Table

Route Table ID: rtb-0af715a925135f5e7

Routes:

10.0.0.0/16 → local

Security Groups

Bastion Security Group

SG ID: sg-0fa3f62b328c17dd1

Name: BastionSecurityGroup

Inbound Rules:

SSH (22) from 0.0.0.0/0

HTTP (80) from 0.0.0.0/0

Outbound Rules:

All traffic to 0.0.0.0/0

Private EC2 Security Group

SG ID: sg-0f87a9ab9e3446f89

Name: PrivateEC2SecurityGroup

Inbound Rules:

SSH (22) from sg-0fa3f62b328c17dd1

HTTP (80) from sg-0fa3f62b328c17dd1

TCP (8080) from sg-0fa3f62b328c17dd1

Outbound Rules:

All traffic to 0.0.0.0/0

Default Security Group

SG ID: sg-0d40cd050a496c00e

Name: default

Inbound Rules:

All traffic from self

Outbound Rules:

All traffic to 0.0.0.0/0

EC2 Instances

Private EC2 Instance (Docker Web Server)

Instance ID: i-0cc8ed9c410bb40ad

Instance Type: t3.micro

AMI: ami-0bf00b639cffdb8c3

Subnet: subnet-076cb98c769125b0b (Private)

Private IP: 10.0.2.189

Security Group: sg-0f87a9ab9e3446f89

Key Pair: temp

State: running

Launch Time: 2025-11-21 20:00:55 UTC

IAM Profile: EC2S3AccessProfile

Bastion/Public Instance

Instance ID: i-0873c489247351079

Instance Type: t3.micro

AMI: ami-09e7fb5d565f22127

Subnet: subnet-0ed1137d5873610be (Public)

Private IP: 10.0.1.176

Public IP: 16.52.125.17 (via EIP)

Key Pair: temp

State: running

Launch Time: 2025-11-21 20:00:55 UTC

Elastic IPs

Allocation ID	Public IP	Associated With	Private IP
eipalloc-058df360e7cbd71e3	16.52.125.17	i-0873c489247351079	10.0.1.176
eipalloc-01e9693328ce106fc	15.157.230.103	NAT Gateway	10.0.1.16

S3 Buckets

my-app-bucket-xyz12345

The diagram illustrates an AWS Cloud architecture for a 3-tier application, organized into an **Availability Zone**.

External Services:

- DNS or** `http://16.52.125.17` (Icon: 53)
- users** and **devices** (Icon: People and laptop)

AWS Cloud Components:

- VPC:** CIDR: 10.0.0.0/16
 - Public subnet:** 10.0.1.0/24
 - Bastion host: Attached to IGW, public IP enabled
 - IG:** Allows inbound SSH only to the bastion.
 - ALB:** Bastion Host
 - NACL:** 0.0.0.0/0 → IGW
 - Route table:**

172.16.0.0
172.16.1.0
172.16.2.0
 - Private subnet:** 10.0.2.0/24
 - Application server: No public IP, outbound only via NAT Gateway
 - NAT:** Allows private EC2 to make outbound calls (S3, yum updates, etc.)
 - EC2:**
 - Name: AppServer
 - Private subnet: 10.0.2.189
 - No public IP (cannot be accessed from internet directly)
 - IAM Role: EC2S3AccessRole
 - NACL:** 0.0.0.0/0 → NAT
 - Route table:**

172.16.0.0
172.16.1.0
172.16.2.0
- Security group:**
 - Auto Scaling group**
 - EC2 instance contents:**
 - Docker file
 - Springboot app
 - Mongodb

- VPC FLOW LOGS** (Icon: Document with arrow)
- my-app-bucket-xyz12345** (Icon: Bucket)
- S3** (Icon: Bucket)
- IAM** (Icon: Lock)

Connections:

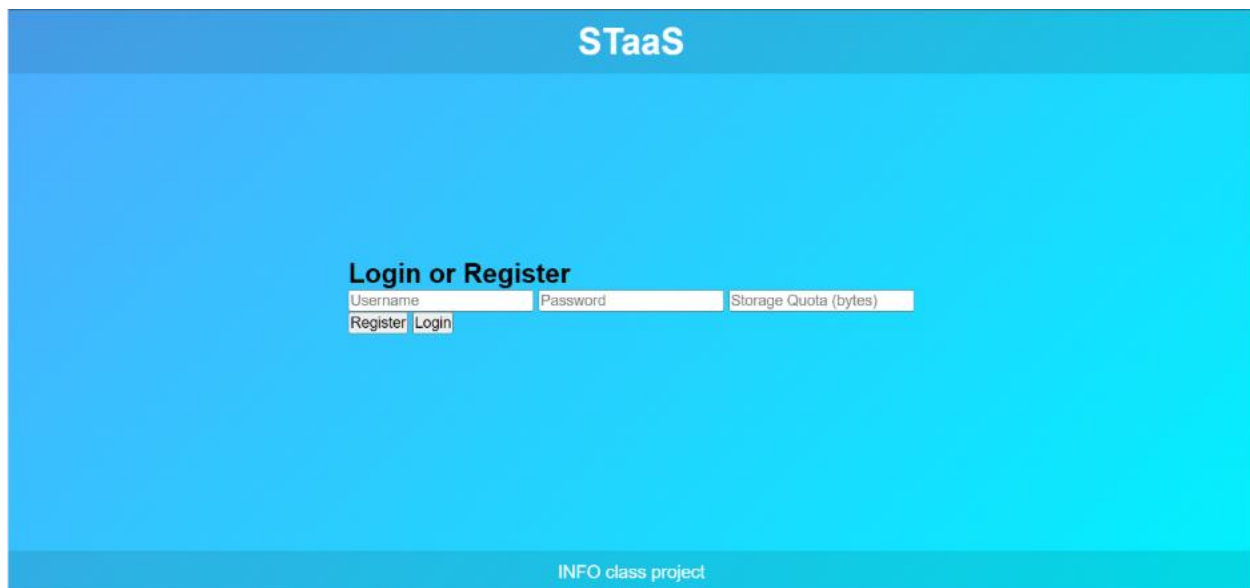
- DNS or `http://16.52.125.17` connects to the Bastion host.
- users and devices connect to the Bastion host.
- The Bastion host connects to the Application server via the NAT Gateway.
- The Application server connects to the S3 bucket via the NAT Gateway.
- The Application server connects to the S3 bucket via the IAM role.
- The Application server connects to the S3 bucket via the VPC FLOW LOGS.

This infrastructure follows a typical public-private subnet architecture:
Public Subnet hosts the bastion instance with direct internet access via IGW.
Private Subnet hosts the Docker web server with outbound internet access via NAT Gateway.
Security Groups enforce least-privilege access between layers.
Elastic IPs provide stable public addresses for both instances.

5. Testing

Login/Register Page

The landing page provides user interface with register and login functionality. The user is given the option to choose storage; by default, the storage is 10000 bytes.



The screenshot displays the 'STaaS' web application interface. At the top, a dark blue header bar contains the text 'STaaS' in white. The main body of the page has a light blue background. Centered on this background is a white rectangular box with the title 'Login or Register' in bold black text. Below the title, there are three input fields: 'Username', 'Password', and 'Storage Quota (bytes)'. The 'Storage Quota (bytes)' field has a default value of '10000'. Below these fields are two buttons: 'Register' and 'Login'. At the bottom of the page, a dark blue footer bar contains the text 'INFO class project' in white.

STaaS		
Login or Register		
Username	Password	Storage Quota (bytes)
Register	Login	
INFO class project		

Dashboard

The dashboard provides users an intuitive interface to interact with the application, featuring drag-and-drop functionality for file uploads and easy access to drive contents through a clear folder and file view.

The dashboard features a light blue header with the text "Welcome, infoproject1" and a "Logout" button. Below this is a status bar showing "Used: 0 / Quota: 1000000 bytes". The main content area includes a "Create Folder" button next to a text input field containing "info4110". Below this is a file upload section with a "Folder name for upload" input, a "Choose File" button, a "no file selected" status, a dashed box with the text "Click or drag files here", and an "Upload File" button. Further down are controls for renaming and deleting: a "Rename File" button with inputs for "Folder name", "Old file name", and "New file name"; a "Delete Folder" button with a "Folder name to delete" input; and a "Delete File" button with inputs for "Folder name" and "File name". A "Refresh Drive" button is located below these controls. A message "Folder created: info4110" is displayed. At the bottom, a "Drive" section shows a folder named "info4110" with the text "No files in this folder."

Uploading New File

The user can upload the file by selecting the folder first and using the choose file radio option to browse or to use the drag and upload feature.

The screenshot displays a file management interface with a light blue background. At the top, it says "Welcome, infoproject1" with a "Logout" link. Below this is a storage status bar: "Used: 135966 / Quota: 1000000 bytes". The main area contains several input fields and buttons. A "Create Folder" button is next to an input field containing "info4110". Below this, there's a "Choose File" button, a file name input field with "Screensh...5.53 PM", a dashed box with "Click or drag files here", and an "Upload File" button. Further down, there are fields for "Folder name", "Old file name", and "New file name" with a "Rename File" button. Below that is a "Folder name to delete" field with a "Delete Folder" button. At the bottom of the main area, there are fields for "Folder name" and "File name" with a "Delete File" button, and a "Refresh Drive" button. A message states "File uploaded successfully: Screenshot 2025-11-04 at 12.05.53 PM.png". At the bottom, a "Drive" section shows a folder "info4110" containing a file "screenshot 2025-11-04 at 12.05.53 pm.png (png, 135966 bytes)" with a "Download" button.

Welcome, infoproject1
[Logout](#)

Used: 135966 / Quota: 1000000 bytes

info4110 [Create Folder](#)

info4110 [Choose File](#) ☒ Screensh...5.53 PM [Click or drag files here](#) [Upload File](#)

Folder name Old file name New file name [Rename File](#)

Folder name to delete [Delete Folder](#)

Folder name File name [Delete File](#)

[Refresh Drive](#)

File uploaded successfully: Screenshot 2025-11-04 at 12.05.53 PM.png

Drive

info4110

- screenshot 2025-11-04 at 12.05.53 pm.png (png, 135966 bytes) [Download](#)

Changing File Name

The user can select the folder and file and rename it to desired change.

The screenshot displays a web-based file management interface with a blue header and a light blue background. At the top, it says "Welcome, infoproject1" with a "Logout" link. Below this is a quota bar showing "Used: 135966 / Quota: 1000000 bytes". The main area contains several input fields and buttons. The first row has a text input "info4110" and a "Create Folder" button. The second row has a text input "info4110", a "Choose File" button, a file preview "Screenshot... 5.53 PM", a dashed box with "Click or drag files here", and an "Upload File" button. The third row has a text input "info4110", a file name "screenshot 2025-11-04 at 12.05.53 pm.png", a file name "filescrnsht", and a "Rename File" button. Below this are two rows for deletion: "Folder name to delete" with a "Delete Folder" button, and "Folder name" with "File name" and a "Delete File" button. A "Refresh Drive" button is at the bottom left. A message "File renamed to: filescrnsht" is displayed. At the bottom, a "Drive" section shows a folder "info4110" containing a file "filescrnsht (png, 135966 bytes)" with a "Download" button.

Welcome, infoproject1
Logout

Used: 135966 / Quota: 1000000 bytes

info4110 Create Folder

info4110 Choose File Screenshot... 5.53 PM Click or drag files here Upload File

info4110 screenshot 2025-11-04 at 12.05.53 pm.png filescrnsht Rename File

Folder name to delete Delete Folder

Folder name File name Delete File

Refresh Drive

File renamed to: filescrnsht

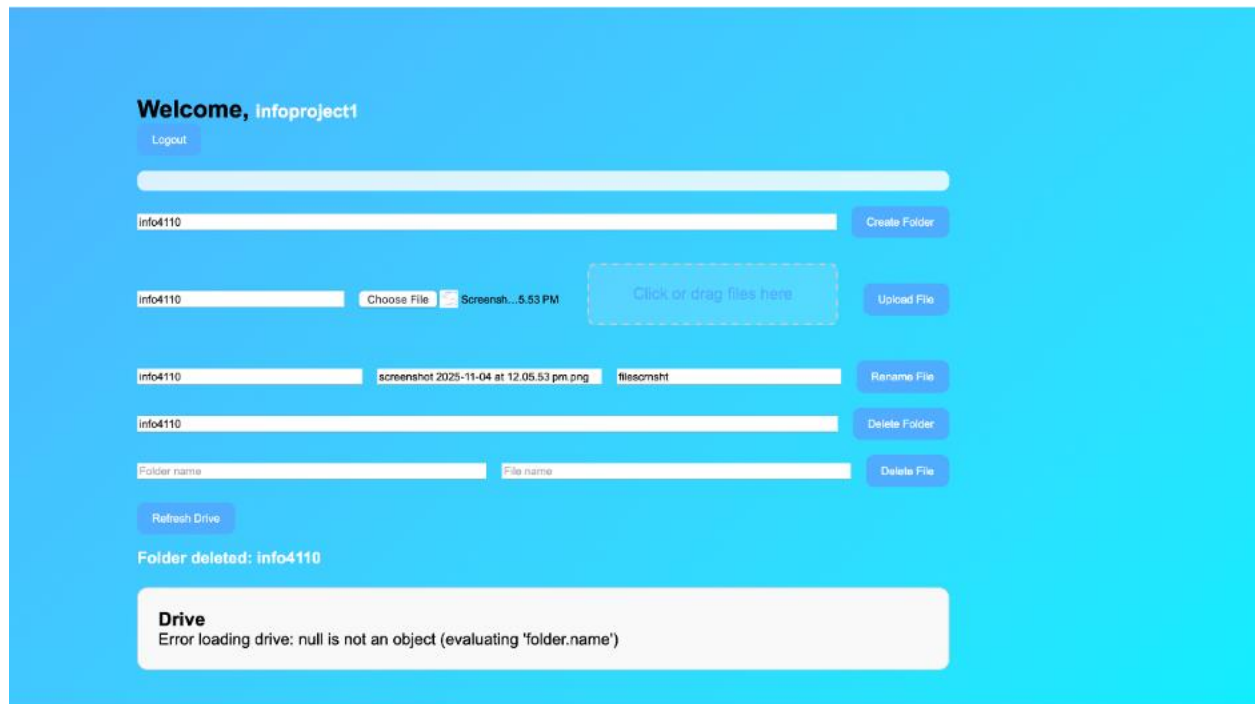
Drive

info4110

- filescrnsht (png, 135966 bytes) Download

Deleting Folder

The application allows users to delete folders, including all contents within them. During testing, a minor bug was found: when the drive is empty, the delete operation returns a null object, which the application does not yet handle.



Deleting File

The user can delete the file from their desired folder.

The screenshot displays the STaaS web interface. At the top, a blue header bar contains the text "Welcome, infoproject1" and "STaaS". Below the header, a light blue bar shows "Used: 0 / Quota: 1000000 bytes". The main content area is light blue and contains several input fields and buttons. A "Logout" button is in the top left. Below the quota bar, there is a text input field with "info4110" and a "Create Folder" button. Further down, there is a text input field with "info4110", a "Choose File" button, a file name "Screenshot 2025-11-21 144429 5.53.PNG", a dashed box with "Click or drag files here", and an "Upload File" button. Below this, there are three text input fields labeled "Folder name", "Old file name", and "New file name", followed by a "Rename File" button. Then, there is a text input field labeled "Folder name to delete" and a "Delete Folder" button. Below that, there is a text input field with "info4110", a text input field with "Screenshot 2025-11-21 144429 5.53.PNG", and a "Delete File" button. A "Refresh Drive" button is located below the "Delete File" button. A message "File deleted: Screenshot 2025-11-21 144429 5.53.PNG" is displayed. At the bottom, there is a white box titled "Drive" containing the text "info4110" and "No files in this folder." The footer of the page is a grey bar with the text "infocj class project".

Welcome, infoproject1 STaaS

Logout

Used: 0 / Quota: 1000000 bytes

info4110 Create Folder

info4110 Choose File Screenshot 2025-11-21 144429 5.53.PNG Click or drag files here Upload File

Folder name Old file name New file name Rename File

Folder name to delete Delete Folder

info4110 Screenshot 2025-11-21 144429 5.53.PNG Delete File

Refresh Drive

File deleted: Screenshot 2025-11-21 144429 5.53.PNG

Drive

info4110

No files in this folder.

infocj class project

6. Timelines, Resource Allocation & Project Planning

6.1 Project Timeline

Phase	Duration
Requirement Analysis	Week 1
System Design	Week 2
Backend Development	Week 3-4
Frontend Development	Week 5
Testing & Debugging	Week 6
Final Deployment	Week 7

The project timeline was carefully crafted to ensure a seamless development process and timely delivery of the STaaS system. It began with the Requirement Analysis phase, where the core objectives, system features, and technical constraints were clearly defined.

Following this, the System Design phase focused on outlining the system architecture, structuring the database, and devising a comprehensive cloud integration strategy.

In the Backend Development phase, secure RESTful APIs were developed using Spring Boot, while MongoDB was integrated for efficient metadata storage, authentication, and authorization management. The Frontend Development then focused on creating a responsive and intuitive user interface to facilitate file and folder management.

Once development was complete, rigorous Testing & Debugging ensured the system's stability, security, and overall functionality. The final phase, Deployment, concluded with a successful system launch and the preparation of detailed documentation.

This structured approach helped manage time effectively, minimized errors, and ensured that each project phase-development, testing, and deployment-was completed thoroughly before moving to the next.

6.2 Resource Allocation

Resource	Purpose
Laptop	Development
AWS S3	File Storage
Mongo DB	Database
Docker	Containerization
IDE (IntelliJ/ VS Code)	Coding

The STaaS project required effective utilization of hardware and software resources for its success. The personal computer or laptop was employed as the main development environment for coding, testing, and debugging the application. Development tools such as IntelliJ IDEA and Visual Studio Code were used to efficiently produce both front-end and back-end code.

Cloud resources played an important role in the project. AWS S3 was used for secure and scalable file storage, while MongoDB was chosen as the primary database to manage users, folders, and file metadata. Docker was used for containerizing the back-end services to enable consistent deployment across different environments.

These resources are picked carefully in order to balance performance, security, scalability, and cost-effectiveness, ensuring that the system operates efficiently under real-world cloud conditions.

7. Summary

The STaaS (Storage as a Service) project successfully demonstrates a secure, scalable, and cloud-based file storage platform. With the incorporation of modern technologies like Spring Boot, MongoDB, AWS S3, and Docker, it essentially offers a real-world cloud storage implementation. The system supports user authentication, folder management, secure file uploads/downloads, and quota enforcement. In addition, this project reflects the real industry-level design of cloud applications.

8. References

Amazon Web Services. (n.d.). *AWS SDK for Java 2.x developer guide*. Retrieved November 5, 2025, from <https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/home.html>

Amazon Web Services. (n.d.). *Boto3 documentation: AWS SDK for Python*. Retrieved November 22, 2025, from <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>

Red Hat. (n.d.). *IaaS vs. PaaS vs. SaaS: What's the difference?* Red Hat. Retrieved October 5, 2025, from <https://www.redhat.com/en/topics/cloud-computing/iaas-vs-paas-vs-saas>

Google Cloud. (n.d.). *What is cloud storage as a service?* Retrieved December 5, 2025, from <https://cloud.google.com/discover/what-is-cloud-storage-as-a-service>